

thuRsdays R Workshop

Lecture 1c: Data – They are not just tables anymore

Venki Uddameri, Ph.D., P.E.

Basic Concepts of R

R was designed to be an interactive programming environment. In other words, analysts work with R to perform tests, refine them and use R as a **tool** to perform their analysis.

R was designed to provide **reproducible workflows**. This means somebody else should be able to perform the same steps with your data and come up with same results. A reproducible workflow not only means the results are reproduced, but all the steps used to obtain the data are clear and transparent.

A reproducible workflow not only describe the final state of your data, but also how you got there

Original	After my Excel analysis
10	10
17	17
999	<i>(deleted)</i>
22	22
NA	<i>(deleted)</i>

If you look at the above Excel spreadsheet you can see some data like 999, NA were deleted. However, the reason behind this deletion is not there. Did I delete the 999 because it is an outlier? Why NA was deleted (wrong data format?). What you will see is the final state but not how we got there. **Reproducible analysis seeks to avoid this ambiguity**

R Analysis

R uses functions to perform analysis. A function is a set of instructions that perform a specific analysis.

For example the function **seq(1,10,1)** will generate a sequence of values between 1 and 10 (both included) in steps of 1

R provides several basic functions. However, these are only the icing on the cake.

R functionality is greatly enhanced by **external libraries**. **External libraries can be found at [CRAN – Comprehensive R Archive Network](#); [Bioconductor](#)** (for bioinformatics); [Github](#).

R provides tools to write your own functions. **You can write these function in R**. These are called **user-defined functions**.

R also provides you tools to take functions written in C, C++, Java, Rust, FORTRAN, Python and other languages and wrap them to run in R. – **Mixed Language Programming**

Language	Typical R interface	Comment
C	<code>.C()</code> , <code>.Call()</code>	Native, long-standing R interface
C++	Rcpp	Probably the most widely used external-language interface
Fortran	<code>.Fortran()</code>	Native interface; important in numerical/scientific computing
Java	rJava	Connects R with the JVM
Rust	extendr, rextendr	Increasingly useful for writing fast, safe extensions
Python	reticulate	Allows Python modules/objects/functions to be used from R
Julia	JuliaCall	Allows R to call Julia

Many functions used to perform matrix calculations, process images and work with genomic data are written other languages and ported into R.

You download libraries on your hard-drive. When create a connection with a particular library using the library(package) function. For example, `library(zoo)` creates a connection with a library called zoo that is downloaded onto your hard-drive.

Finding Libraries

R has thousands of libraries and new libraries are added every day. So trying to memorize which library to use is futile. However, R has a very strong and supportive user community. Therefore, a quick websearch or use of Generative AI can help you identify relevant packages.

Scripts

A script is a set of instructions in R that perform a specific analysis. A script usually has the following elements:

1. Make connection to Libraries that need to be loaded (optional)
2. Functions to load the data (usually required)
3. User-defined functions (UDF) for analysis (optional)
4. Functions to explore and visualize data
5. Functions to perform analysis
6. Function to write output to hard-drive

A script helps make your analysis reproducible, as it lists steps that are used in the analysis along with the initial and output data.

Knowing Which Functions to Use:

There are thousands of libraries and each library may have hundreds of functions. Therefore, the number of available functions is large. Therefore, trying to memorize which functions to use is also futile. However, a quick web search or careful prompting with Gen AI can get you to the required set of functions fairly quickly.

While there are hundreds and thousands of functions, we often use a very small subset based on what we do. In time, these functions will become second nature. You will also get better and finding new functions or even write your own functions.

Some R Quirks

As a programming language R has its own syntax structure. This might not be in line with what you have experienced. Here are some unique characteristics of R

1. Everything in R is an object. Data, variables, functions, libraries are all objects.
2. R is case-sensitive (r and R mean different objects)
3. R uses <- as an assignment operator for example a <-3 means R is assigning a value of 3 to object a. (3 is stored in the memory belonging to object a).
4. R uses forward slash "/" rather than backward slash "\" when you define your file paths and folders. The use of "\" is actually a quirk of windows (linux and Mac both use / to define path). So define your path using / rather than \ in R scripts.
5. R has 1 based indexing. Which means vectors, matrices, lists, tables are start with 1. Python on the other hand uses 0 based indexing.
6. The use of Brackets is clearly defined in R:
 - a. A function is always followed by parenthesis – for example a <- c(3.4,1,3,2.)
 - b. Square brackets are used to access elements of a vector, list or a matrix. For example a[1] gives me 3.4
 - c. A list uses double [[]] double square brackets. For example b <- list("a",2,4) then b[[1]] gives me "a"
 - d. Curly brackets are used to define code blocks. Anything between {} will be executed as a group.

7. R is an interpretive language. Which means, the program executes line by line (unless there are loops – in which case the entire loop is completed as a single block before moving to the next line).

Gen AI – A Crutch or an Assistant

Gen AI tools like ChatGPT, Claude and others can write R code. However, the quality of the code written is a function of your prompts. The clearer and focus the prompts are, the better is the code. However, following points need to be borne in mind:

1. The code is not always correct. Or is suboptimal.
2. The Gen AI might give you outdated information as R libraries are dynamic and improve and change as they adopt to technologies (in most cases, Gen AI is trained on older versions)
3. If you run the analysis on Gen AI, the workflow is no longer reproducible.
4. Uploading data on to Gen AI platforms may violate data dissemination policies or make your data public (even before your analysis is published).

You need to know some basic syntax of R and have an idea of your data to use Gen AI carefully.

Recommendations on using Gen AI for R

- Query R for functions and libraries but do not ask it write your whole script.
- Rather than ask Gen AI to run the analysis, put the code in your script and see what results it produces.
- Ask Gen AI to work with synthetic data rather than uploading your own data. Gen AI is very good at generating synthetic datasets
- Ask Gen AI to explain any syntax errors you encounter when writing your scripts. You can paste the error and prompt Gen AI to tell you what it is.
- If you encounter R code written by others, you can paste snippets in Gen AI (assuming you are allowed to do so) and ask it to explain the code.